

BVR RL in DCS World

A step-by-step guide: from a computer without DCS to a trained 1v1 policy flying an F-16C against the DCS AI.

BVR_RL · branch feature/dcs-1v1 @ 267b75a · 9 October 2026

The live link described here has been tested against a stand-in for DCS (our own simulator speaking the same protocol, and a mock of DCS's scripting interface). It has not yet been run inside DCS itself.

Contents

Part 1 · Before you start

1. What you will build	4
2. What you need, and the seven steps	5

Part 2 · Step by step

3. Step 1 · Install DCS World	7
4. Step 2 · Get the code and test without DCS	7
5. Step 3 · Let DCS missions talk to Python	8
6. Step 4 · Make the mission	9
7. Step 5 · Shadow run: the policy only watches	10

8. Step 6 · Let the policy fly	11
9. Step 7 · Read the results	12
10. Fly against the policy yourself (free)	13

Part 3 · When things go wrong, and reference

11. Troubleshooting	16
12. How DCS differs from training	16
13. What the bridge does to the DCS AI	17
14. Commands and options	18
15. Protocol and files	20
16. Undoing everything	21

How to use this guide

- Do Part 2 in order. Each step ends with a green **Done when** box: don't go on until you see what it describes.
- Each instruction is marked with where you do it: **DCS** in DCS World, **TERMINAL** in a normal terminal in the BVR_RL folder, **ADMIN TERMINAL** in a terminal opened with *Run as administrator*.
- Commands are shown in grey boxes; what the computer answers is shown in lighter boxes below them. Numbers in the answers are examples: yours will differ.
- If something fails, Part 3 lists the symptoms seen so far and what to do.

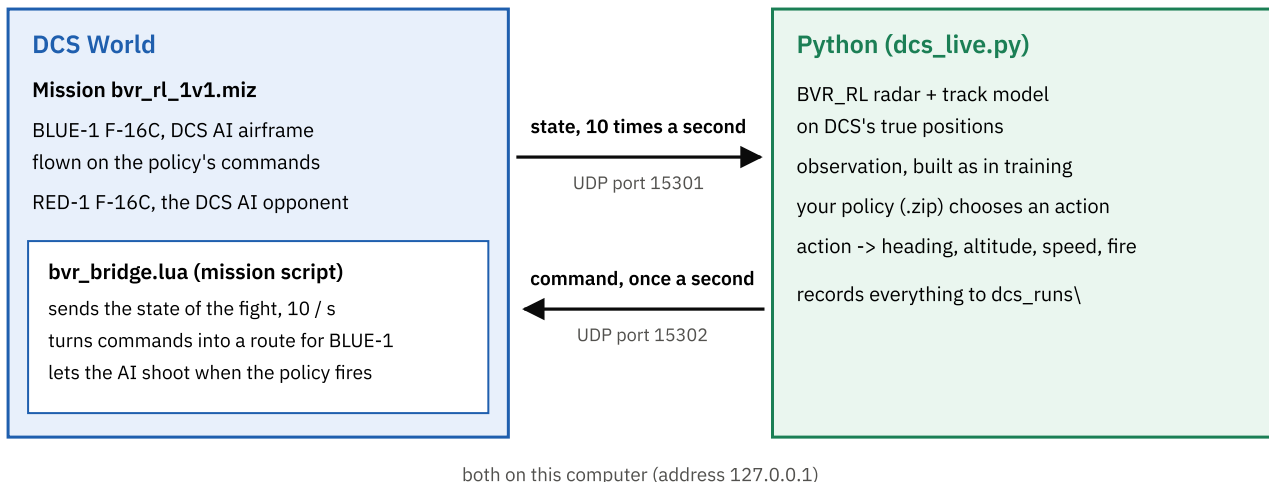
PART 1

Before you start

What the pieces are, how they talk to each other, and what you need on your computer.

1 · What you will build

Your policy was trained in BVR_RL's own simulator. Here it flies in DCS World instead: DCS flies the aircraft and the missiles, and the policy decides, once a second, where to point the aircraft and when to shoot.



The two halves and what goes between them. Nothing leaves your computer.

One second of a fight

- 1 DCS reports.** Ten times a second the bridge sends where every aircraft and air-to-air missile is, which way it points and how fast it moves, plus events: shots, hits, deaths, missiles left.
- 2 Python builds what the policy sees.** BVR_RL's own radar and track model runs on those true positions, exactly as in training, and the 68 observation inputs are built from the track. DCS's radar is not used.
- 3 The policy chooses.** A heading offset from the line to the bandit, an altitude change, one of five speeds, and fire or not: the same action as in training.
- 4 Python sends a command.** The action becomes an absolute heading, altitude and speed, and a fire flag.
- 5 DCS flies it.** The bridge gives BLUE-1's AI a route along the commanded heading, at the commanded altitude and speed. On "fire" it lets the AI launch one AIM-120 at RED-1, then takes control back.

The files

File	What it does
<code>dcs/bvr_bridge.lua</code>	The mission script. Packed into the mission; you never run it yourself.
<code>dcs/setup_dcs.py</code>	One-time patch that lets mission scripts use a network socket (Step 3).
<code>dcs/make_mission.py</code>	Builds the mission file <code>bvr_rl_1v1.miz</code> (Step 4).
<code>dcs_live.py</code>	Runs your policy against DCS, or only watches with <code>--shadow</code> (Steps 5 and 6).
<code>dcs/fake_dcs.py</code>	Pretends to be DCS, using BVR_RL's simulator, to test without DCS (Step 2).
<code>dcs_obs_check.py</code>	Compares the inputs the policy got in DCS with the ones it got in training (Step 7).
<code>dcs/README.md</code>	The same instructions in short form.

2 · What you need, and the seven steps

Checklist

You need	Notes
A Windows 10 or 11 PC	DCS World runs on Windows only. Python and DCS run on the same computer.
Disk space on an SSD	DCS is a large download, well over 100 GB with the free maps. Check the current requirements on the DCS website before you start.
Memory and graphics	In practice 16 GB of RAM is the minimum and 32 GB is better; a dedicated graphics card.
The Python environment you train in	The one that runs <code>train_bvr.py</code> and the GUI. One more package, <code>pydcs</code> , is added in Step 2.
A trained 1v1 checkpoint	Any <code>.zip</code> from <code>models_bvr\</code> trained in 1v1, ideally as the F-16C against the F-16C (the platforms the mission uses). 2v1 checkpoints are refused.
Administrator rights	Once, for Step 3: DCS is installed under <code>C:\Program Files</code> .
Nothing to buy	Both aircraft are DCS AI units, which are free for every type. A paid module is only needed to fly the red aircraft yourself.

The seven steps



Step	What you do	Why
1	Install DCS World and fly a free mission once.	DCS itself works.
2	Get the <code>feature/dcs-1v1</code> branch; fly your policy against the fake DCS.	The Python half and the network link work, before DCS is involved.
3	Run <code>setup_dcs.py</code> as administrator.	DCS lets mission scripts talk to Python.
4	Run <code>make_mission.py</code> , copy the mission.	A 1v1 mission with the bridge inside.
5	Start <code>dcs_live.py --shadow</code> , then the mission.	The link works with DCS; you see how the policy reads a DCS fight, without it flying.
6	Start <code>dcs_live.py</code> , then the mission.	The policy flies BLUE-1.
7	Read <code>dcs_runs\</code> ; run <code>dcs_obs_check.py</code> .	Outcomes, and where DCS differs from training.

Why the order matters

Each step adds one new thing. If Step 2 works and Step 5 doesn't, the problem is on the DCS side (the patch, the mission, the bridge), not in Python. If Step 5 works and Step 6 doesn't, it is in how the bridge steers or fires, not in the link.

PART 2

Step by step

From installing DCS to reading the results of a policy flying in it.

3 • Step 1 • Install DCS World

- 1 **WEB** Go to digitalcombatsimulator.com and make an account (*Sign up*, top right). Confirm it from the e-mail they send.
- 2 **WEB** On the same site open *Downloads* → *DCS World* and download the installer (the "standalone" version).
DCS is also on Steam as *DCS World Steam Edition*. Either works; this guide uses the standalone folders. With Steam, DCS is under ...\\steamapps\\common\\DCSWorld, and `setup_dcs.py` looks there too.
- 3 Run the installer. Keep the default folder, `C:\\Program Files\\Eagle Dynamics\\DCS World`, unless that disk lacks space; an SSD makes a large difference. The installer downloads the game, which can take hours.
- 4 **DCS** Start DCS from the desktop icon and log in with your account. The first start is slow while it prepares its graphics.
- 5 **DCS** Fly once to be sure it works: *Instant Action* → a free aircraft (Su-25T or TF-51D) → any mission → *Fly*. Press `Esc` → *Quit* to leave.

What is free, and why it is enough

The free game includes the Caucasus map and two aircraft you can fly yourself (Su-25T, TF-51D). What matters here: **every aircraft type can be placed in a mission as an AI unit without buying it**, including the F-16C. The policy's aircraft and its opponent are both AI units: the policy steers its aircraft through the AI. Buy a module (such as the F-16C) only if you want to fly a fair fight against the policy yourself; Chapter 10 shows how to fly against it for free, as a defender in the Su-25T.

Done when DCS starts, you are logged in, and an Instant Action mission flies.

4 • Step 2 • Get the code and test without DCS

Before DCS is involved, check the Python half: `fake_dcs.py` pretends to be DCS (using BVR_RL's own simulator, speaking exactly the bridge's protocol) and `dcs_live.py` flies your policy against it.

- 1 **TERMINAL** Open a terminal in the BVR_RL folder, with your training environment active, and get the branch:

```
git fetch
git checkout feature/dcs-1v1
pip install pydcs
```

- 2 **TERMINAL** Start the policy. Use any 1v1 checkpoint you have:

```
python dcs_live.py models_bvr\\latest.zip
```

```
policy: models_bvr\\latest.zip (F-16C v F-16C), BALANCED doctrine
listening for DCS on UDP 15301, commands to 15302
waiting for DCS (start the mission that runs bvr_bridge.lua) ...
```

- 3 **TERMINAL 2** Open a second terminal in the same folder and start the fake DCS:

```
python dcs\fake_dcs.py --speed 4
```

--speed 4 flies four seconds of fight per real second.

- 4 Watch the first terminal. Every 10 seconds it prints the range, the track state and the policy's choices, FIRE where it shoots, and the outcome at the end:

```
episode 1: BLUE-1 v RED-1, start range 91.6 km
t= 10s range 87.3 km track TRACK heading +30 alt +0 speed 340
t= 41s range 38.0 km track TRACK heading +30 alt +0 speed 340 FIRE
episode 1: KILL after 88 s, 1 shots (0 fire requests not answered by a launch)
written: dcs_runs\live_20261002_181500_ep1.jsonl, _steps.csv
results: KILL 1
```

If Windows asks whether Python may use the network, allow it on *private networks*. Everything stays on this computer (address 127.0.0.1).

Done when the first terminal shows an episode from start to outcome, and `dcs_runs\` holds a `.jsonl`, a `_steps.csv` and `results.csv`. From here on, anything that fails is on the DCS side.

5 · Step 3 · Let DCS missions talk to Python

DCS takes networking and file access away from mission scripts, for safety. The bridge needs one thing back: **LuaSocket**, a small network library that ships with DCS. `setup_dcs.py` adds a few lines to DCS's file `Scripts\MissionScripting.lua` that load LuaSocket before DCS locks the scripts down, and keep it under one name, `bvr_rl_socket`. Everything else (files, the operating system, loading other code) stays locked.

- 1 **DCS** Close DCS completely.
- 2 **ADMIN TERMINAL** Open a terminal as administrator: Start menu, type `cmd`, right-click *Command Prompt* → *Run as administrator*. Activate your Python environment in it if you use one, and go to the BVR_RL folder (`cd C:\...\BVR_RL`).
- 3 **ADMIN TERMINAL** Run:

```
python dcs\setup_dcs.py
```

```
DCS World: C:\Program Files\Eagle Dynamics\DCS World
LuaSocket: found
MissionScripting.lua: not patched
backup: C:\Program Files\Eagle Dynamics\DCS
World\Scripts\MissionScripting.lua.bvr_rl_backup
patched. Restart DCS if it is running. Run this again after every DCS update.
```

If you see	Do
DCS World not found	Give the folder: <code>python dcs\setup_dcs.py --dcs "D:\Games\DCS World"</code> (the folder that holds <code>bin</code> and <code>Scripts</code>).
permission denied	The terminal is not running as administrator. Open it again as in item 2.

LuaSocket: NOT FOUND	Patch anyway; if Step 5 then reports that LuaSocket is missing, tell me the DCS version and what is in the DCS folder.
already patched	Nothing to do.

Know what the patch allows

- With the patch, **any** mission you open can make network connections, not only this one. Undo it when you stop using BVR_RL (python dcs\setup_dcs.py --undo, as administrator), and don't join multiplayer servers with it in place: some check this file and turn you away.
- Every DCS update puts the original file back. Run setup_dcs.py again after each update. --check tells you whether the patch is there, without changing anything.

Done when python dcs\setup_dcs.py --check reports MissionScripting.lua: patched.

6 · Step 4 · Make the mission

1 **TERMINAL** Build the mission:

```
python dcs\make_mission.py
```

```
written dcs\bvr_rl_1v1.miz: BLUE-1 (F-16C, policy) v RED-1 (F-16C, DCS AI), 90 km apart
copy it to Saved Games\DCS\Missions\ (or open it from anywhere in DCS)
```

pydcs may first print *Couldn't detect any installed DCS World version* many times. Ignore it: the mission is still written.

2 Copy dcs\bvr_rl_1v1.miz into C:\Users\<you>\Saved Games\DCS\Missions\.

What is in the mission

- The Caucasus map, over open sea west of Sukhumi, so there is no terrain in the fight.
- **BLUE-1**: an F-16C (AI, skill Excellent) with four AIM-120C, as the training platform carries. The policy flies it. Without dcs_live.py it flies a combat air patrol and fights on its own.
- **RED-1**: an F-16C (AI) with four AIM-120C, 90 km away, flying towards BLUE-1, 500 m higher. The bridge orders it to attack BLUE-1.
- A trigger that starts bvr_bridge.lua when the mission starts. The script is packed inside the .miz.
- No player aircraft: DCS opens the mission in the map and external views.

Variations

Option	Effect
--range-km 70	Start separation (default 90; training starts are 70–110 km).
--bearing 45	Direction from blue to red, degrees from map north (default 90).
--alt-blue 8000 --alt-red 10000	Start altitudes in metres (defaults 9000 and 9500).
--speed 250	Start speed, m/s (default 280).
--red-type Su-27	Red's aircraft: F-16C, F-15C, Su-27, MiG-29S, FA-18C, Su-25T. Tell dcs_live.py with --opp-platform which library platform's missile envelopes to assume for it.
--red-player	You fly RED-1 (needs that module).

<code>--defender</code>	You fly RED-1 in the free Su-25T and defend (Chapter 10).
<code>--out my.miz</code>	Another file name, to keep several missions.

Building it by hand instead

In the Mission Editor (main menu → *Mission Editor* → *New*, map Caucasus): place an **F-16C bl.50** for blue (unit name **BLUE-1**, task CAP, 9000 m, four AIM-120C, a second waypoint towards red) and one for red (**RED-1**, 70–110 km away, flying towards blue). Then *Triggers*: new trigger of type **4 MISSION START**, action **DO SCRIPT FILE** → `dcs\bvr_bridge.lua`. Save into Saved Games\DCS\Missions\. The bridge flies the first blue airplane and fights the first red one.

Done when `bvr_rl_1v1.miz` is in Saved Games\DCS\Missions\.

7 • Step 5 • Shadow run: the policy only watches

In a shadow run nothing is sent to DCS. DCS's own AI flies BLUE-1 and fights; the policy sees everything and works out what it would do. This checks the link with real DCS data, and shows how the policy reads a DCS fight, with no effect on the fight.

- 1 **TERMINAL** Start the policy in shadow mode (a normal terminal, not administrator):

```
python dcs_live.py models_bvr\latest.zip --shadow
```

```
waiting for DCS (start the mission that runs bvr_bridge.lua) ...
```

- 2 **DCS** Start DCS. Main menu → *Mission* → choose `bvr_rl_1v1` → *Fly*. If the mission starts paused, press **Pause**.

- 3 **DCS** At the top right of the screen the bridge says:

```
bvr_bridge: BLUE-1 v RED-1, sending to 127.0.0.1:15301, listening on 15302
```

There is no player aircraft, so you see the map (**F10**). **F2** shows an external view of an aircraft; press it again for the next one.

- 4 **TERMINAL** The terminal starts the episode and prints it every 10 seconds: the range, the track, and the heading, altitude and speed the policy *would* choose, with FIRE where it would shoot.

```
episode 1: BLUE-1 v RED-1, start range 89.7 km [shadow: the DCS AI flies, the policy
only watches]
t= 10s range 83.1 km track TRACK heading +30 alt +0 speed 340
```

- 5 When the episode ends the terminal prints the outcome and the files it wrote.

What to look at

- The **range** in the terminal should match the F10 map (the ruler tool measures it).
- The **track** should reach TRACK as the aircraft close, as in training.
- The policy's choices are what it would do in a fight DCS's AI is flying, so they may look odd: the aircraft is not where the policy would have put it.

Done when the terminal shows an episode with a track and an outcome, and `dcx_runs\` holds `live_..._shadow.jsonl`.

8 • Step 6 • Let the policy fly

- 1 **TERMINAL** Start the policy without `--shadow`:

```
python dcs_live.py models_bvr\latest.zip --speed-boost
```

- 2 **DCS** Start the mission as in Step 5.

- 3 From the first command the bridge flies BLUE-1:

- the DCS AI's own reactions to threats are switched off and its weapons held, so the policy decides when to turn, defend and shoot;
- every second the policy's heading, altitude and speed become a route for the AI to fly;
- on FIRE the bridge lets the AI shoot one AIM-120 at RED-1, then takes control back;
- RED-1 is the DCS AI, ordered to attack BLUE-1.

- 4 **TERMINAL** The terminal shows the fight and its end:

```
t= 52s range 41.2 km track TRACK heading +30 alt +0 speed 340 FIRE
t= 60s range 36.0 km track TRACK heading +50 alt +0 speed 340
episode 1: KILL after 97 s, 1 shots (0 fire requests not answered by a launch)
written: dcs_runs\live_20261002_181500_ep1.jsonl, _steps.csv
```

- 5 When the episode ends (a kill, a loss, or the step limit) BLUE-1 is given back to the DCS AI, which carries on with its own mission.

Several episodes

Start with `--episodes 5`. After each episode restart the mission in DCS (`Esc` → *Quit* → *FLY AGAIN* in the debriefing). `dcs_live.py` notices the restart and starts the next episode by itself. `Ctrl + C` in the terminal stops it at any time and hands BLUE-1 back to the AI.

Faster fights

DCS time acceleration works: `Ctrl + Z` faster, `Alt + Z` back to normal. Python handles about 8 seconds of fight per real second on an ordinary computer. If it falls behind it says so:

```
this computer is 2.4 s of mission time behind DCS: the policy's commands arrive late (lower
DCS time acceleration)
```

Useful options

Option	Use
<code>--max-steps 400</code>	If you trained with 400-second episodes. The policy sees elapsed time as a share of this, so match training.
<code>--doctrine CONSERVATIVE</code>	The shot doctrine the policy is told (AGGRESSIVE, BALANCED, CONSERVATIVE), as on the WATCH page.

`--opp-platform ...` The library platform whose missile envelopes the policy assumes for red, when red is not an F-16C.

Done when an episode ends with an outcome and BLUE-1 visibly followed the policy (its turns in DCS match the heading choices in the terminal).

9 · Step 7 · Read the results

Everything is in `dc_runs\`:

File	Contents
<code>results.csv</code>	One row per episode: live or shadow, the aircraft, outcome, flight time, start range, shots, fire requests, and requests that never became a launch.
<code>live_..._ep1.jsonl</code>	Everything DCS sent during the episode, in <code>bvr_logger.lua</code> 's format. <code>dc_obs_check.py</code> reads it.
<code>live_..._ep1_steps.csv</code>	One row per second: the action (heading offset, altitude change, speed, fire), the command sent to DCS, the track state, the true range, and all policy inputs. Opens in Excel.

Checking the inputs

The most useful check: did the policy get inputs in DCS like the ones it trained on?

```
python dcs_obs_check.py dcs_runs\live_20261002_181500_ep1.jsonl --platform F-16C --opp-platform F-16C --max-steps 300
```

It replays the recording through the observation code, flies 20 simulator episodes for reference, and prints a table, one row per input, with the DCS and simulator ranges. It writes `report.txt`, `summary.csv` and `dc_inputs.csv` to `dc_check\`. Several recordings can be given at once.

Flag	Means
OUTSIDE	More than 20% of the DCS steps fall outside the range the simulator produced: the policy has rarely or never seen those values.
CLIP	DCS pushes the input past its normalisation limit at least 5 percentage points more often than the simulator: the policy sees it pinned at the limit.

Each flag is one of two things: a **bridge mistake** (a unit, sign or axis error), which should be fixed, or a **real difference** between DCS and the simulator (speeds, turn rates, missile ranges), to decide on: accept it, or retrain with the simulator moved closer to DCS.

What to send me after the first runs

The terminal output, `results.csv`, one `.jsonl` from a shadow run and one from a live run, and `Saved Games\DCS\Logs\dc.log` if anything went wrong. That is enough to find what needs adjusting.

10 · Fly against the policy yourself (free)

You fly RED-1 in the **Su-25T**, which comes free with DCS. It is a ground-attack jet: no air-to-air radar, and only two short-range heat-seeking R-73s. You can't shoot back beyond visual range, so you **defend**. The question this answers: can the policy shoot down a human who knows a missile is coming and evades it?

- 1 **TERMINAL** Build the defender mission and copy it to `Saved Games\DCS\Missions\`:

```
python dcs\make_mission.py --defender
```

```
written dcs\bvr_rl_defender.miz: BLUE-1 (F-16C, policy) v RED-1 (Su-25T, you), 90 km apart
```

It puts you in RED-1 at 6000 m and 200 m/s (the Su-25T is slow: about 950 km/h at most), 90 km from BLUE-1, flying towards it. BLUE-1 is unchanged: an F-16C with four AIM-120Cs. The mission's briefing screen repeats the essentials.

- 2 **TERMINAL** Start the policy, for several fights:

```
python dcs_live.py models_bvr\latest_selfplay_v6.zip --episodes 5 --speed-boost
```

- 3 **DCS** *Mission* → `bvr_rl_defender` → *Fly*. You start in the cockpit, in the air. Keep time acceleration at 1×.
- 4 Survive. BLUE-1 will turn towards you and shoot from long range.
- 5 After each fight: `Esc` → *Quit* → *FLY AGAIN*. The terminal starts the next episode by itself.

Defending

- **Watch the radar warning receiver.** It shows BLUE-1's radar. It may warn of a missile only late, when the missile's own radar switches on, so treat a radar lock as a possible launch.
- **Beam the missile:** turn so the threat is at your 3 or 9 o'clock. A radar missile loses its target when the target moves across its line of sight.
- **Release chaff** while beaming, and **descend**. Find the countermeasures key in *Options* → *Controls*, Su-25T, "Countermeasures".
- **Or turn away early**, so the missile has to chase you and runs out of energy. You can't outrun the F-16 itself.
- If the policy lets you get within a few kilometres, your R-73s can hit it.

New to the Su-25T?

DCS has simplified "game" flight and avionics modes in *Options* → *Gameplay* that make it easier to fly. Practise once in an Instant Action mission first.

Reading the result

Outcome in the terminal	Means
KILL	The policy shot you down.
TIMEOUT	You survived until the time limit (300 s by default): your win.
ESCAPE	You got more than 150 km from BLUE-1.
SHOT_DOWN	You shot BLUE-1 down with an R-73.
BANDIT_CRASH	You crashed or ejected.

`python dcs\inspect_run.py dcs_runs\live_..._ep1.jsonl` shows when each missile was fired, at what range, and how it ended.

What the policy doesn't know

- It only knows the F-16C, so it **assumes you carry AIM-120s**. It will defend against shots you can't make, and keep more distance than it needs.
- A slow Su-25T at low altitude is outside anything it trained on.

So read these fights as a check of how it tracks, shoots and follows up against a human who evades, not as a fair score.

PART 3

When things go wrong, and reference

Symptoms and fixes, what to expect from DCS, what the bridge changes in the DCS AI, and every command option.

11 • Troubleshooting

Errors inside the bridge go to Saved Games\DCS\Logs\dcsl.log, as lines containing bvr_bridge. Search the file for that word first.

Symptom	Cause and fix
No bvr_bridge: ... message when the mission starts	The trigger is missing, or the script stopped with an error. Check dcs.log. If you built the mission by hand, check the MISSION START trigger and its DO SCRIPT FILE action.
bvr_bridge: LuaSocket is not available	The patch is missing: run setup_dcs.py as administrator (Step 3) and restart DCS. After every DCS update, run it again.
bvr_bridge: need a blue and a red airplane	The mission lacks one of them, or names given in CFG.agent / CFG.red don't exist.
bvr_bridge: cannot listen on port 15302	Another program uses the port. Change port_in at the top of bvr_bridge.lua, rebuild the mission, and start dcs_live.py with --port-out set to the same number.
Python keeps printing waiting for DCS	DCS isn't sending: look for the bridge message; make sure the mission isn't paused. A firewall that blocks Python on <i>private</i> networks can also stop it.
cannot listen on UDP port 15301 in Python	Another dcs_live.py (or fake_dcs.py test) is still running. Close it.
fire timeout in the terminal	The DCS AI did not launch within 8 s. Its radar may not have RED-1 yet, or RED-1 is outside the AI's own launch range. If it never fires, open bvr_bridge.lua, set fire_weapon = nil (the AI picks the missile itself), and rebuild the mission.
BLUE-1 turns and climbs more slowly than in training	The DCS AI flies the route its own way. This is a real difference (Chapter 12), not a fault.
... behind DCS	Lower the time acceleration (Alt + Z).
The next episode doesn't start	Restart the mission in DCS; dcs_live.py waits for mission time to go back to the start.
this checkpoint was trained for 2v1	The DCS link is 1v1 only for now. Use a 1v1 checkpoint.
Multiplayer server refuses you	The patched MissionScripting.lua. Run setup_dcs.py --undo before joining.

12 • How DCS differs from training

Expect the policy to do worse in DCS than in training at first. These are the reasons, and where to see each one.

Difference	What it means	Where to see it
------------	---------------	-----------------

Flying	BLUE-1 is flown by the DCS AI along the commanded route, with DCS's F-16 flight model. Turns, climbs and speed changes differ from the training autopilot. A DCS AI on a route does not light its afterburner for the speed it is asked for (BLUE-1 stayed at 99% RPM, never above 280 m/s; RED-1, in combat, reached 490). Fly with --speed-boost: DCS is then asked for 550 m/s while BLUE-1 is slow, and does use the afterburner, mostly to climb, which comes close to the simulator's F-16 with afterburner. Train as F-16C-DCS. --eta-lock made no difference. Turns are the bigger gap: on the route the AI banks about 45° and pulls about 1.5 g, about 2°/s, where the simulator's F-16 turns at about 8°/s, so every crank, beam and turn away takes four times longer than the policy expects. dcs/turn_test.py measured it (chapter 14): with the first route point 3 km, 1 km or not at all ahead, every turn was flown at exactly 45° of bank and 1.41 g, 1.5–1.6°/s at 340 m/s. The bridge cannot make it turn harder. Climbs are flown at about Mach 0.85 whatever speed is asked for, at about 0.02 m/s per metre still to climb (50–60 m/s at 10 km with 3 km to go), where the simulator's F-16 climbs with afterburner and accelerates. The platform F-16C-DCSAI climbs and turns the DCS AI's way (at most 45° of bank): train the agent on it, with red on F-16C-DCS (self-play still works, as the two differ only in how they climb and turn). Defending is where 2°/s hurts most, so since bridge version 4 the DCS AI can defend BLUE-1 itself: with --auto-defend (on by itself for a checkpoint trained on F-16C-DCSAI-AD) the bridge hands BLUE-1 to the DCS AI's own missile defence once a missile aimed at it is within 15 km, and back to the policy when it is gone. In the recorded fights the DCS AI defended exactly that late: nose-on until the missile was 13.4–13.5 km away, then a hard break toward the beam (90–170° of bank, diving 1–3 km). F-16C-DCSAI-AD does the same in training, fitted to those defences and drawn anew each time (from 12–15 km, 3–4.5 g, 70–110° off the missile, a 1.5–4 km dive, no shots). Turns toward the enemy can be fast too: an AI given an attack task turns at about 7°/s (80–94° of bank, 4 g; turn_test.py --variants hot, 9 October). Since bridge version 5, with --hot-turn (on by itself for F-16C-DCSAI-AD), a commanded heading within 15° of RED-1's bearing, with BLUE-1 more than 30° off it, is flown as a guns-only attack until BLUE-1 is within 10° of it. F-16C-DCSAI-AD flies those turns at 77° of bank in training (6.9°/s). Turns away from RED-1 stay at 1.6°/s.	The terminal's speed : line; speed_cmd_mean, speed_flown_mean, speed_short_pct in results.csv.
Shooting	"Fire" is a request. The AI launches when its own logic agrees (set to shoot at maximum range), usually within a second or two. No second shot is allowed until the launch is seen.	results.csv: fire timeouts.
Missile support	In training a missile misses after 3 s without guidance from its shooter's radar track, until its own seeker takes over; since SIM_REV 9 this holds for scripted opponents too. DCS is more forgiving. By default dcs_live.py applies the training rule to both aircraft (red's track comes from the same radar model run from red's side on DCS's positions) and has the bridge remove a missile that loses support; --support-rule dcs leaves it to DCS.	Terminal; missiles_removed in results.csv.
Radar	The track comes from BVR_RL's radar model on DCS's true positions, as in training. DCS's radar is not used, so this part matches training by design.	—
Opponent	RED-1 is the DCS AI, which none of the training opponents imitate exactly. Its defence (beam and dive to near the sea) is closest to ADAPTIVE's deep defence (SIM_REV 10). DCS also models ground clutter, so a low, beaming target can break a missile's lock there; training has no clutter.	Outcomes; the .json1 replay.
Missiles	DCS's AIM-120 reaches differently from the envelope tables the policy trained with.	Shot ranges vs outcomes.

13 · What the bridge does to the DCS AI

Nothing changes until the first command arrives, which is why a shadow run leaves the fight to the DCS AI.

When	What the bridge sets
Mission start	RED-1, if it is AI: weapons free, and an attack task on BLUE-1.
First command	BLUE-1: weapons hold ; reaction to threats none ; radar continuous search ; missile attack range maximum ; no jettison.
Each command	A two-point route from BLUE-1's position along the commanded heading (3 km and 60 km out) at the commanded altitude and speed. Re-issued only when the command moves by 2° of heading, 50 m or 1 m/s, or every 10 s.
Fire	Weapons open fire , and an attack task on RED-1 for one active-radar missile. While it waits, the route is not changed.
Launch seen (or 8 s without one)	Weapons hold again, the attack task removed, the route re-issued. Python is told launched or timeout .
Support lost (DESTROY)	The missile is removed in DCS: a miss, as in training. Python is told destroyed (or gone if it had already ended).
End of episode (STOP)	BLUE-1's tasks cleared; reaction to threats back to evading fire; weapons free. It carries on as an ordinary AI aircraft.

Settings at the top of `bvr_bridge.lua`

Change them in the file, then rebuild the mission (the mission keeps its own copy of the script).

Setting	Default	Meaning
port_out / port_in	15301 / 15302	UDP ports to and from Python
rate	0.1 s	Time between state messages
agent / red	nil	Unit names; nil means the first blue / red airplane
red_attack	true	Order an AI red aircraft to attack the agent
near_m / far_m	3000 / 60000	Distances of the two route points
reissue_deg / reissue_s	2 / 10	When to re-issue the route
fire_timeout	8 s	Wait for a launch before giving up
fire_weapon	134217728	DCS weapon flag for the shot: active-radar air-to-air. nil lets the AI choose
resend_s	2 s	How often the header and ammunition are repeated, for a late-starting Python

14 · Commands and options

`dcslive.py`

```
python dcs_live.py CHECKPOINT [options]
```

Option	Default	Meaning
--------	---------	---------

--shadow	off	Never send commands: the DCS AI flies, the policy only watches
--episodes N	1	Episodes before quitting; restart the mission between them
--max-steps N	300	Episode length in seconds; match training
--doctrine D	BALANCED	Shot doctrine the policy is told
--opp-platform P	checkpoint's	Library platform assumed for red's envelopes
--platform P	checkpoint's	Library platform the policy is told it flies (e.g. F-16C-DCS; F-16C-DCS-MIL is the F-16C without afterburner, as DCS flies BLUE-1 without --speed-boost; F-16C-DCSAI climbs like the DCS AI)
--eta-lock [C]	off	Give the route's points locked arrival times from the commanded speed (said to make the DCS AI use afterburner; in our tests it made no difference). C: mission (default) or abs (time of day).
--auto-defend	platform's	Once a missile aimed at BLUE-1 is within 15 km, the DCS AI defends it at its full agility (beam, dive, chaff) and the policy may not fire; the policy flies again when the missile is gone. On by itself for a platform with AUTO_DEFEND (F-16C-DCSAI-AD). Needs bridge version 6 (4–5 took over at the launch). <code>results.csv: defences, defend_s</code>
--hot-turn	platform's	A heading command toward RED-1 (within 15° of its bearing), with BLUE-1 more than 30° off it, is flown as a guns-only attack, about 7°/s instead of the route's 1.6, until BLUE-1 is within 10° of it. On by itself for a platform with HOT_TURN_BANK (F-16C-DCSAI-AD). Needs bridge version 5. <code>results.csv: hot_turns, hot_turn_s</code>
--speed-boost [V]	off	While BLUE-1 is more than 20 m/s below the policy's speed, ask DCS for V m/s (default 550) instead; the real speed again once within 5 m/s. This makes the DCS AI use its afterburner: use it whenever the policy was trained with afterburner
--support-rule R	training	training: 3 s without support before the seeker takes over removes the missile (blue's and red's); dcs: DCS decides
--agent / --red	from bridge	DCS unit names, if not the bridge's choice
--port-in / --port-out	15301 / 15302	UDP ports
--out DIR	dcs_runs	Output folder

dcs/make_mission.py

Option	Default	Meaning
--range-km	90	Start separation
--bearing	90	Direction blue → red, degrees from map north
--alt-blue / --alt-red	9000 / 9500	Start altitudes, m
--speed	280	Start speed, m/s
--blue-type	F-16C	F-16C, F-15C, FA-18C. Keep F-16C for an F-16C policy
--red-type	F-16C	F-16C, F-15C, Su-27, MiG-29S, FA-18C, Su-25T
--red-player	off	You fly RED-1

--defender	off	You in the free Su-25T, defending; writes <code>bvr_rl_defender.miz</code>
--speed-red	--speed	Red's start speed, m/s (200 for the Su-25T)
--out	<code>dcx\bvr_rl_1v1.miz</code>	Mission file

dcx/setup_dcs.py (as administrator)

Option	Meaning
(none)	Find DCS, back up and patch <code>MissionScripting.lua</code>
--check	Report only
--undo	Remove the patch; the file is restored exactly
--dcs PATH	DCS install folder, if not found

dcx/fake_dcs.py

Option	Default	Meaning
--speed	4	Fight seconds per real second
--opponent	SHOOTER	Scripted red: STRAIGHT, EVASIVE, NOTCHER, SHOOTER, ADAPTIVE_SHOOTER
--platform / --opp-platform	F-16C	Library platforms
--missions	1	Missions one after another (use with <code>dcx_live.py --episodes</code>)
--seed	0	Start geometry

dcx/turn_test.py

Flies BLUE-1 through 90° right, 90° back and 170° right, no policy, for each of four ways of building the route, while RED-1 holds its fire, and reports how fast it turned. Needs a mission with bridge version 3 (rebuild it with `make_mission.py`). About 15 minutes of mission time for all four. The fifth variant, `hot` (bridge version 4, only when asked for), puts RED-1 90° off a wing on the route, then gives BLUE-1 an attack task on it with guns only and measures the turn to it: does an attacking AI turn harder than one on a route? It stops before RED-1 is within 25 km. Measured on 9 October: 7.0–7.1°/s, 80–94° of bank, 4.1 g, where the route turns at 1.6°/s. The sixth, `hotturn` (bridge version 5), does the same by commanding the heading to RED-1 and leaving the attack to the bridge, as `dcx_live.py --hot-turn` does. At the end of a test RED-1 still holds its fire and BLUE-1 flies on along its last route: end the mission.

Option	Default	Meaning
--variants	the four route ones	base (first point 3 km ahead, as <code>dcx_live.py</code>), <code>near1000</code> (1 km), <code>far_only</code> (no first point), <code>flyover1000</code> (1 km, fly-over points), <code>hot</code> (an attack task on RED-1)
--speed / -alt	340 / start	The flight condition
--boost	550	As <code>dcx_live.py --speed-boost</code> ; 0 turns it off
--sim	–	The same turns in BVR_RL's simulator, through <code>fake_dcs.py</code> , for comparison

For each turn: the turn rate while more than 30° is still to go, the time to half and to 90% of the turn, the bank and the g. One row per turn is added to `dcx_runs\turn_test.csv`. With bridge version 3, keep a run under

about 10 minutes: at bingo fuel (16%) the DCS AI flies home and ignores the route. Version 4 turns that off for both aircraft.

dcs_obs_check.py

Option	Default	Meaning
RECORDING ...	—	One or more .jsonl files
--platform / --opp-platform	F-16C	The platforms the policy was trained with
--max-steps	300	Training episode length
--ref-episodes / --ref-opponent	20 / SHOOTER	The simulator reference
--out	dcs_check	Output folder

Tests (no DCS needed)

python test_sim.py	(includes "DCS live link": dcs_live.py against fake_dcs.py over UDP)
pip install lupa	
python dcs\test_bridge.py	(bvr_bridge.lua and the setup patch, in Lua 5.1)

15 • Protocol and files

DCS → Python (UDP 15301)

One JSON object per message: the same lines bvr_logger.lua writes (state samples, shots, hits, deaths, ammunition), plus:

{ "format": "bvr_rl.dcs.v1", ... , "bridge": 6, "agent": "BLUE-1", "red": "RED-1" }	every 2 s
{ "ev": "fire", "t": ..., "status": "requested launched timeout refused", "seq": ... }	
{ "ev": "bridge", "t": ..., "status": "control released", "agent": ... }	
{ "ev": "bridge", "t": ..., "status": "eta", "clock": ... }	(OPT eta)
{ "ev": "bridge", "t": ..., "status": "opt", "near_m": ..., "wpt": ..., "redhold": ..., "autodefend": ..., "hot": ... }	(any OPT but eta)
{ "ev": "bridge", "t": ..., "status": "defend", "on": true false }	(OPT autodefend: the DCS AI takes over / hands back)
{ "ev": "bridge", "t": ..., "status": "hotturn", "on": true false }	(OPT hotturn: a turn toward red flown as an attack / the route again)
{ "ev": "support_lost", "t": ..., "id": ..., "status": "destroyed gone failed" }	
{ "ev": "kill", "t": ..., "unit": ..., "killer": ..., "id": ... }	
{ "ev": "mission_end", "t": ... }	

Python → DCS (UDP 15302)

```
CMD <seq> <heading deg, map north, clockwise> <altitude m> <speed m/s> <fire 0|1> <mission  
time, ignored by the bridge>  
STOP  
DESTROY <missile id>  
OPT eta <off|mission|abs>  
OPT near <m>                first route point this far ahead (0: none)  
OPT wpt <turn|flyover>      route points' type  
OPT redhold <0|1>           an AI red holds its fire  
OPT autodefend <0|1>        the DCS AI defends BLUE-1 while a missile is in flight at it  
OPT hot <0|1>               an attack task on red, guns only (turn_test.py)  
OPT hotturn <0|1>           turns toward red flown as such an attack
```

The header's bridge is the version: 2 understands OPT eta, 3 OPT near, wpt and redhold, 4 OPT autodefend and hot, 5 OPT hotturn, 6 OPT autodefend only within 15 km (4–5: from the launch); since 4 neither aircraft flies home at bingo fuel. While BLUE-1 defends (or attacks), fire requests are refused with reason defending (hot).

Commands are re-sent every second, so a lost message costs at most a second.

Where things are

What	Where
DCS	C:\Program Files\Eagle Dynamics\DCS World
The patched file and its backup	...\DCS World\Scripts\MissionScripting.lua, ... \MissionScripting.lua.bvr_rl_backup
Missions	C:\Users\<you>\Saved Games\DCS\Missions\
DCS's log	C:\Users\<you>\Saved Games\DCS\Logs\dcs.log
Run results	BVR_RL\dcs_runs\
Input check results	BVR_RL\dcs_check\

16 · Undoing everything

- ADMIN TERMINAL** Remove the patch: `python dcs\setup_dcs.py --undo`. The file is put back exactly as it was. (The backup `MissionScripting.lua.bvr_rl_backup` stays next to it; delete it if you like.)
- Delete `bvr_rl_1v1.miz` from `Saved Games\DCS\Missions\`.
- Delete `dcs_runs\` and `dcs_check\` in the `BVR_RL` folder, if you don't need the recordings.
- `git checkout feature/platforms-2v1` takes the code back to the branch without the DCS link.